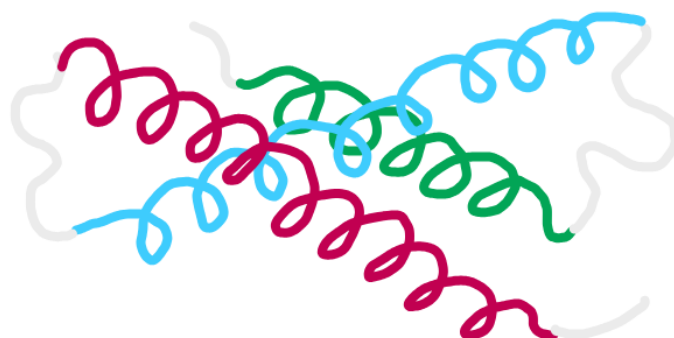




Rysunek 1: Jedyne białko jakie zwiniliśmy (UPF0410 protein YmgE)

Zwijanie białek - MimNet

Metody Probabilistyczne w Uczeniu Maszynowym

Szymon Szulc, Konrad Ciborowski

16 czerwca 2025

Spis treści

1	Wstęp	2
2	Motywacja	2
3	Problem zwijania białek	2
4	Zbiór danych	3
5	Mimetic Neural Network	3
6	Implementacja (teoria)	4
6.1	Wejście i wyjście	4
6.2	Krok w czasie	4
6.3	Funkcja f	4
6.4	Funkcje straty	5
6.4.1	Zwijanie	5
6.4.2	Projektowanie	6
6.4.3	Regularyzacja	6
6.4.4	Łączona	6
6.5	Synteza	6
7	Implementacja (praktyka)	6
8	Wyniki	6
9	Rysunki	8
10	Alternatywne podejście do equilibrium	9
11	Podsumowanie	10

1 Wstęp

Niniejszy raport oparty jest na repozytorium Feq149/mim_net. Raport ma stanowić zwięzłe i czytelne podsumowanie naszej pracy nad problemem predykcji zwinięcia białka (PSP), korzystając z sieci MimNet.

2 Motywacja

Bezpośrednią przyczyną zainteresowania jest gra FoldIt. Polega ona na manipulowaniu strukturą białka w taki sposób, żeby osiągnęło stan equilibrium energetycznego. Okazuje się, że problem zwijania białek jest dla nas ludzi bardzo istotny, FoldIt regularnie organizuje konkursy, żeby zwinąć daną sekwencję aminokwasów. Najlepsze wyniki są przekazywane badaczom, żeby mogli sprawdzić zwinięcie empirycznie i odkryć jakie funkcjonalności ma dane białko.



Rysunek 2: Gra FoldIt

3 Problem zwijania białek

Zwijanie białek to proces, w którym wytworzone białko przyjmuje swoją specyficzną, trójwymiarową strukturę. Ta forma jest kluczowa dla jego prawidłowego funkcjonowania w organizmie. Dzięki informacjom o zwijaniu możemy przewidywać funkcje białek lub projektować białka o danej funkcjonalności.

Zrozumienie procesu zwijania białek ma ogromne znaczenie dla medycyny, biotechnologii i projektowania leków.

Przewidywanie struktury białka na podstawie jego sekwencji aminokwasowej jest jednym z najtrudniejszych problemów w biologii obliczeniowej. Głównym powodem tej trudno-

ści jest ogromna liczba możliwych kombinacji, jakie może przyjąć pojedynczy łańcuch aminokwasów. Białka nie mogą losowo przeszukiwać wszystkich możliwych kombinacji, przecież zwijają się w ułamku sekundy. Białka wiedzą coś więcej niż my.

W ostatnich latach sztuczna inteligencja zrewolucjonizowała dziedzinę predykcji struktury białek. Największym przełomem było opracowanie AlphaFold2. Model ma dokładność bliską eksperymentalnej. My zajmiemy się prostszym modelem MimNet.

4 Zbiór danych

Korzystamy ze znanego zbioru białek CASP. CASP jest tak naprawdę zbiorem zbiorów, jego kolejne edycje są coraz trudniejsze. Ma on na celu zweryfikowanie, co ludzkość już potrafi, a czego jeszcze nie. My wybraliśmy CASP7 jako zbiór treningowy. Liczy on ≈ 20000 białek. Jako zbiór testowy, korzystamy z gotowego podzbioru *test*, który jest dostępny w CASP7.

5 Mimetic Neural Network

Będziemy bazować na opisie MimNetu z artykułu: Eliasof, M., Boesen, T., Haber, E., Keasar, C., Treister, E. *Mimetic Neural Networks: A Unified Framework for Protein Design and Folding*.

Powyżej widzieliśmy dwa problemy związane ze strukturą białek – zwijanie i projektowanie. Istnieje wiele modeli, które zajmują się tymi problemami, ale rozłącznie. Wydaje się, że te problemy są ze sobą ściśle powiązane. MimNet to model, który korzysta z tego faktu. Jest to sieć, która jest zdolna jednocześnie przewidywać zwinięcie oraz ciąg aminokwasów (projektować), korzystając z tych samych wag, dzięki temu proces uczenia jest wspólny. Jego głównym założeniem jest odwracalność. MimNet czerpie garściami ze świata fizyki, naśladuje system fizyczny, który zmienia się w czasie, ale jest również odwracalny. Autorzy mają sporo racji w takim podejściu, przecież białko to system fizyczny. Jest energia, są oddziaływania na poziomie atomów i pewnie inne czynniki fizyczne.

Warto zaznaczyć, że wejściem do MimNet są statystyki pierwszego rzędu, do których dojdziemy później. Ma to pewnie znaczenie z punktu widzenia dostępności danych.

W tym momencie warto doprecyzować, co my właściwie przewidujemy. Otóż trochę skłamałobyśmy, pisząc, że przewidujemy zwinięcie. My przewidujemy współrzędne (x, y, z) atomów węgla C_α . Jest to węgiel, który łączy się z grupą funkcyjną aminokwasu i okazuje się, że to wystarcza do predykcji zwinięcia. Mając te współrzędne stosuje się następujące kroki szukania equilibrium energetycznego:

1. Losowe przypisanie atomów C_β .

2. Heurystyki dotyczące minimalizacji energii (*wiggle* w FoldIt).
3. Przypisanie reszty atomów w mądry sposób.

6 Implementacja (teoria)

Jeśli nie jest doprecyzowane o jakim problemie piszemy, to jest to problem zwijania – tylko to u nas ”działa” :).

6.1 Wejście i wyjście

Wejściem $[S, S^+]$ do naszej sieci jest ciąg aminokwasów w one-hot-encoding oraz macierz PSSM (z grubsza jest to macierz $20 \times L$, która mówi PSSM $[i, j]$ z jakim prawdopodobieństwem na miejscu j występuje aminokwas i). Wyjściem $\hat{X}$ jest macierz $3 \times L$, która ma bardzo naturalną interpretację – koordynaty (x, y, z) . Dla problemu projektowania następuje zamiana semantyczna wejścia z wyjściem :).

To były nasze prawdziwe wejścia i wyjścia. One sprawiają problemy z odwracalnością, no bo mają zupełnie inne typy, rozmiary. Dlatego autorzy wprowadzają zanurzenia. Dla wejścia jest to Conv1D z jądrem rozmiaru 1 z n_f kanałami wyjściowymi. Efektywnie daje nam to kodowanie każdego aminokwasu jako $v \in R^{n_f}$. Z wyjściem jest podobnie – Conv1D z jądrem rozmiaru 1 i 3 kanałami wyjściowymi. Dla problemu projektowania analogicznie.

6.2 Krok w czasie

Teraz zaczynają się ciekawe rzeczy. Cały model opiera się na tym, że system zmienia się w czasie – nasze zanurzenie zmienia się w czasie. Autorzy z modelu Verleta wyprowadzili następujące równanie:

$$\mathbf{Y}_{j+1} = 2\mathbf{Y}_j - \mathbf{Y}_{j-1} + h^2 f(\mathbf{Y}_j, \boldsymbol{\theta}_j),$$

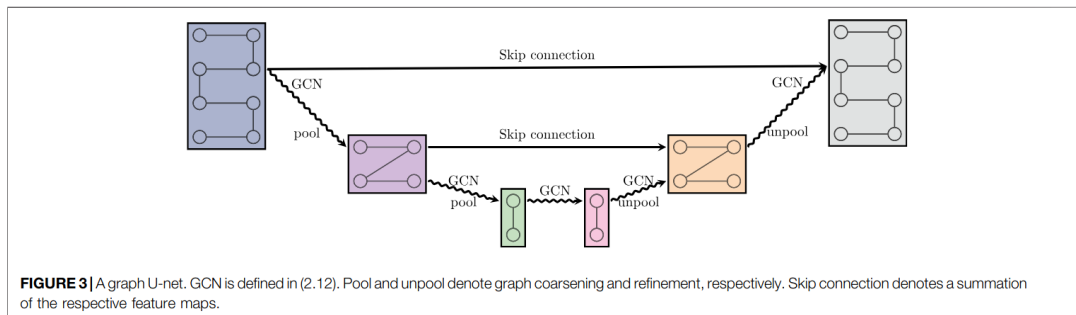
gdzie h to miara czasu – u nas zawsze 1, f - to dowolna funkcja, my będziemy chcieli nią przechwycić informacje o strukturze białka, dlatego naturalnym wyborem będzie jakaś konwolucja grafowa – w końcu białka to też grafy. Dla formalności $\mathbf{Y}_0$ to nasze zanurzenie. W przypadku projektowania $\mathbf{Y}_T$ to będzie zanurzenie odwrotne z $3 \times L$ do $n_f \times L$.

Żeby się cofnąć w czasie zamieniamy $\mathbf{Y}_{j+1}$ i $\mathbf{Y}_{j-1}$ miejscami.

6.3 Funkcja f

Ona ma za zadanie zebrać wszystkie informacje o strukturze białka. Nie możemy użyć zwykłej konwolucji grafowej, przeszkadza jej zakres – jest bardzo lokalna i informacje

z odległych części grafu długo się propagują. Autorzy proponują sieć UNet. Najlepiej działanie wyjaśni obrazek.



Rysunek 3: UNet

Co my robimy:

- Najpierw

$$\text{GCN}_{j+1} = \omega_j \text{GCN}_j + \sigma(\mathcal{N}(K_j(\text{GCN}_j + \text{GCN}_j L_j))),$$

gdzie K_j to konwolucja 1D z jądrem o rozmiarze 9, zbierająca informacje z sąsiadów, a L_j to Laplasjan skonstruowany z $W_j = \exp(-\alpha^{-1} \mathbf{D}(\text{GCN}_j)) - \mathbf{D}$ to operator odległości między aminokwasami w kwadracie, $\alpha = 10$ ustalona empirycznie przez autorów. $\mathcal{N}$ to warstwa normalizacji, a σ to aktywacja ReLU. ω_j równa się 1, gdy nie stosujemy pooling-u grafu, i 0 w przeciwnym wypadku.

- Potem pooling albo unpooling - z grubsza łączmy 2 wierzchołki w 1 średnią albo robimy interpolację liniową.
- I tak w kółko.
- Dodatkowo wprowadzone są połączenia pomijające – charakterystyka UNetu.

Wydaje się to skomplikowane, ale intuicyjnie przyspiesza to zbieranie informacji z wierzchołków zachowując wagi Laplasjanem.

6.4 Funkcje straty

6.4.1 Zwijanie

$$\ell_{\text{fold}} = \frac{1}{n_M} \sqrt{\|M \odot (D(\text{MimNet}(S, \theta)) - D(X))\|_F}$$

To wygląda bardzo strasznie, ale w rzeczywistości – liczymy jak bardzo się pomyliliśmy dla każdego aminokwasu, usuwamy te które są za daleko, żeby miały na siebie wpływ – maska M dostarczona w zbiorze. Bieremy średnią.

6.4.2 Projektowanie

$$\ell_{\text{design}} = \sum S^+ \odot \log \left(\frac{\hat{S}^+}{S^+} \right) + \sum S \odot \log \left(\frac{\hat{S}}{S} \right)$$

To znamy bardzo dobrze – dywergencja Kullbacka-Leiblera.

6.4.3 Regularyzacja

$$R(\theta) = \sum_j \|\theta_{j+1} - \theta_j\|_1$$

To jest norma L_1 z sąsiadujących UNetów, o czym zaraz. Ma to na celu gładkie zmiany w czasie.

6.4.4 Łączona

$$J(\theta) = \frac{1}{N} \sum_j \left[\ell_{\text{fold}}(\text{MimNet}(S_j, \theta), X_j) + \ell_{\text{design}}(\text{MimNet}^\dagger(X_j, \theta), S_j) \right] + \beta R(\theta)$$

6.5 Synteza

Mamy już wszystkie komponenty. Wielka tajemnica zostaje ujawniona. MimNet to T połączonych UNetów + zanurzenia.

7 Implementacja (praktyka)

Nie obyło się bez tarapatów. Ich głównym powodem wcale nie była biblioteka PyTorch, ale szczątkowy opis autorów, nadal nie mamy pewności czy nasz MimNet to MimNet z artykułu. Główne problemy sprawiła implementacja bloku UNet. Pomniejsze problemy sprawiał eksplodujący gradient, z którym szybko sobie poradziliśmy przycinając jego wartości. Ogólne odczucia pozytywne, PyTorch jest bardzo przyjemny do pracy z nim.

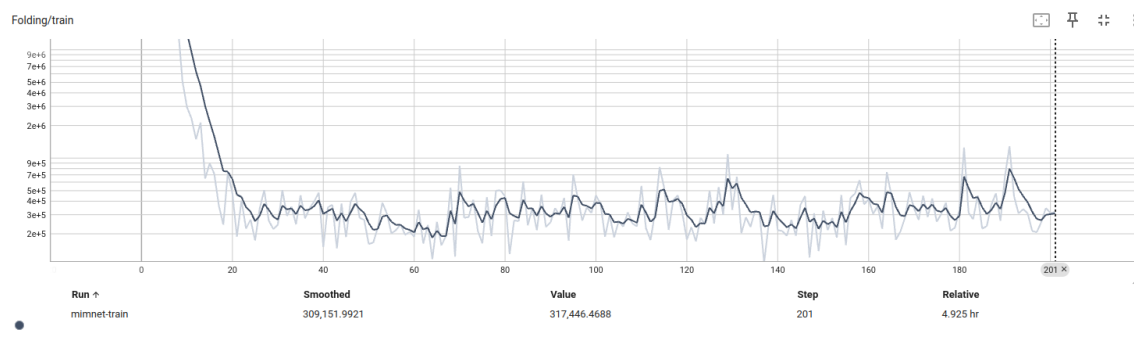
8 Wyniki

Trenowanie odbyło się z następującymi parametrami:

- optymalizator Adam ze współczynnikiem uczenia 0.0001,
- $n_f = 128$,
- $T = 6$,
- $\beta = 0.0001$
- liczba epok 50 (początkowo 200, ale wykresy uczenia pokazały, że po 50 już biegamy),

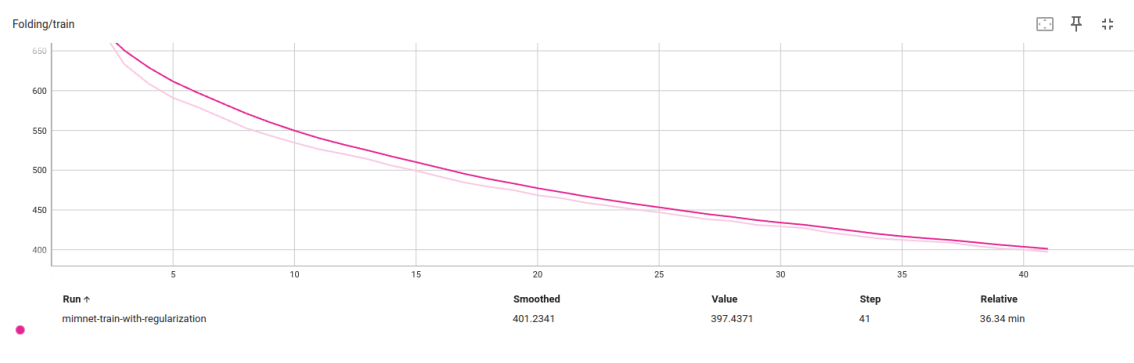
- rozmiar batcha 24 – tyle był w stanie wytrzymać miracle.

Najpierw trening zbugowanego MimNetu. Tutaj była jeszcze zwykła konwolucja grafowa. Jak widać na załączonym obrazku jest słabo. Jeśli dobrze pamiętam nie ma tutaj normalizacji batcha, co jest niezwykle istotne w przypadku sieci grafowych – proszę nie pytać o dowód, fakcik z kursu SI.

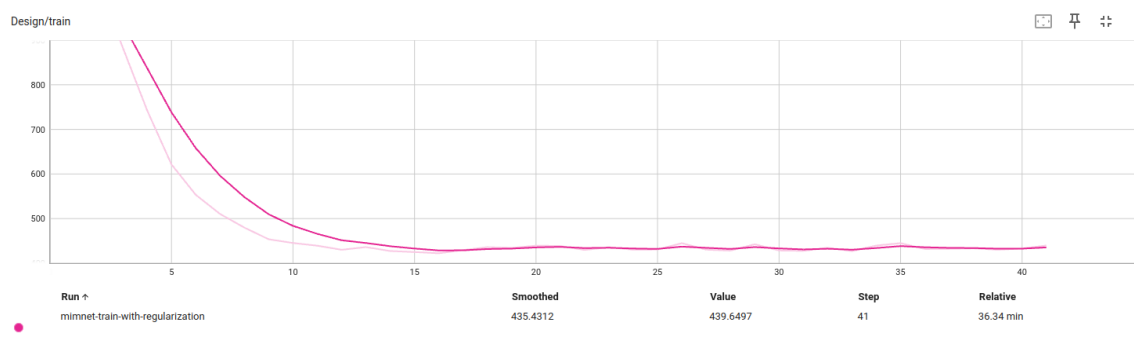


Rysunek 4: Krzywa uczenia dla zwykłej konwolucji grafowej

Wartości funkcji straty są ogromne. Na szczęście udało się zaimplementować UNet ”poprawnie”. Różnice są kolosalne, zeszliliśmy 3 rzędy wielkości.



Rysunek 5: Krzywa uczenia MimNet zwijanie



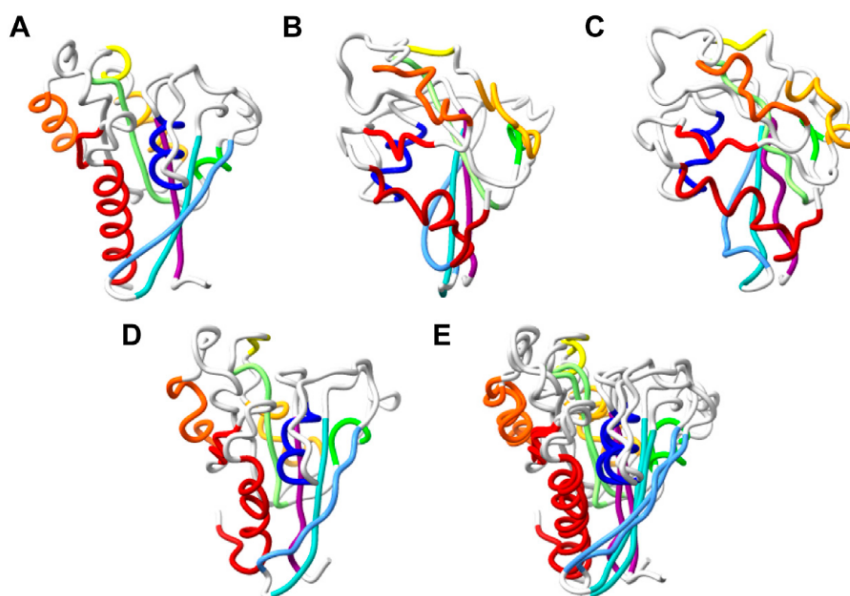
Rysunek 6: Krzywa uczenia MimNet projektowanie

Na te wykresy aż się miło patrzy, gładko zbiegamy do lokalnego minimum. Niestety zbiór testowy zweryfikował, że model źle się uogólnia $\ell_{\text{fold}} = 204474.0327$, $\ell_{\text{design}} = 9377.1319$.

Czas uczenia dla 200 epok to prawie 5 godzin :(.

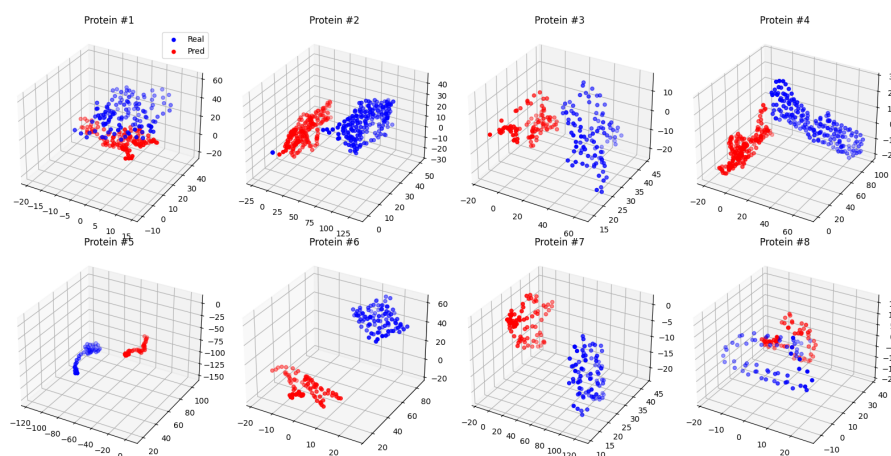
9 Rysunki

Bardzo mnie (Szymon) zasmuciło, że nie damy rady rzeczywiście zwinąć białek jak autorzy.



Rysunek 7: Wyniki zwijania autorów artykułu

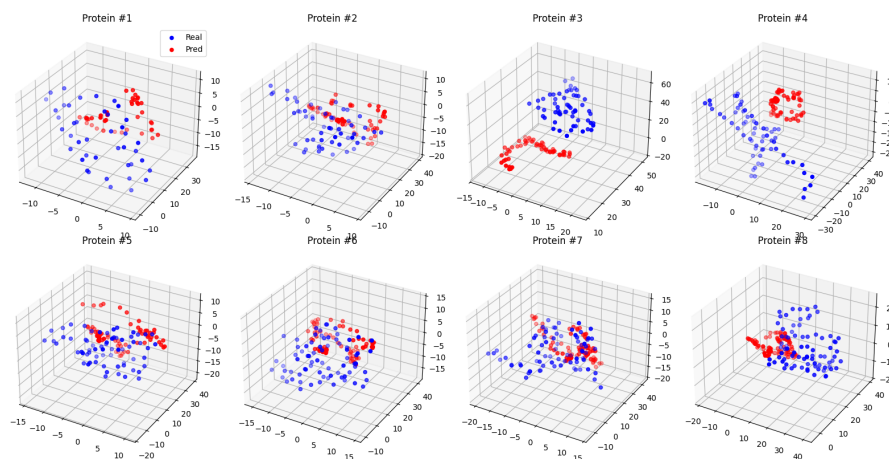
Dlatego wymyśliłem jakiś sposób, żeby to wizualizować. Możemy zrobić wykres punktowy atomów C_{α} . Poniżej ciekawsze wyniki.



Rysunek 8: Nasze wyniki zwijania na zbiorze treningowym

Jak widzimy na wykresach nasz MimNet trochę działa. Czasami te zbiory się pokrywają, czasami są po prostu w innej orientacji. Uznajemy to za sukces :).

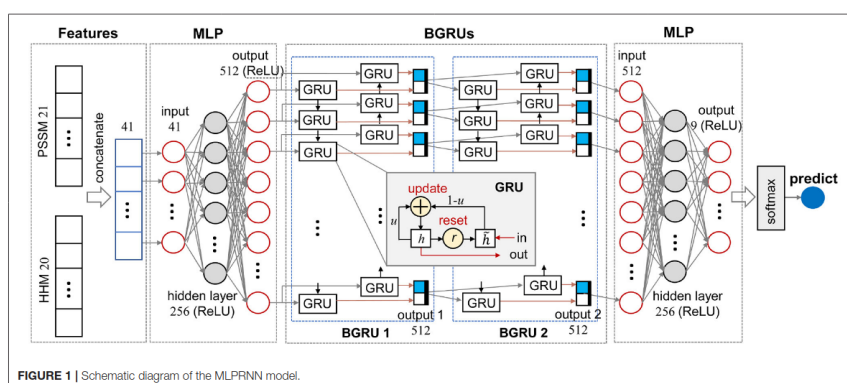
Na zbiorze testowym jest gorzej, ale przynajmniej nie jest to losowe – widać jakieś struktury.



Rysunek 9: Nasze wyniki zwijania na zbiorze testowym

10 Alternatywne podejście do equilibrium

Nie udało się zrealizować przewidywania struktury drugorzędowej – problem klasyfikacji wieloklasowej, chcemy określić do jakiej struktury przestrzennej (α -helisa, β -harmonijka, zwój) będzie należał każdy aminokwas. Co by nam to dało? Pomysł był taki, żeby między punktami w 3D narysować właśnie te struktury. Poniżej zamieszczamy model z artykułu: Lyu, Zhiliang and Wang, Zhijin and Luo, Fangfang and Shuai, Jianwei and Huang, Yandong *Protein Secondary Structure Prediction With a Reductive Deep Learning Method*. Nie będziemy wchodzić w szczegóły, ale możemy o tym porozmawiać na obronie.



Rysunek 10: RNN

11 Podsumowanie

Nie osiągnęliśmy zadowalających efektów, problem struktury białek jest bardzo trudny. Wymaga głębokiego rozumienia, jakie korzyści daje nam dana sieć. My takiego rozumienia nie mamy ;). Na szczęście pojawił się promyk nadziei – wykres punktowy.

Literatura

- [1] Katarzyna Grygiel Wykład z Metod Probabilistycznych w Uczeniu Maszynowym, 2024-2025.
- [2] Eliasof, M., Boesen, T., Haber, E., Keasar, C., Treister, E. Mimetic Neural Networks: A Unified Framework for Protein Design and Folding, 2022.
- [3] Lyu, Zhiliang and Wang, Zhijin and Luo, Fangfang and Shuai, Jianwei and Huang, Yandong Protein Secondary Structure Prediction With a Reductive Deep Learning Method, 2021.